

The Token *Survival Kit*

Ran out of tokens again? This is the fix. 20 field-tested ways to stop hitting the Claude Code wall — plus one master prompt you paste once to set it all up. No fluff, no email gate. Built and edited by Claude Code itself.

01

Start here

5 steps from zero to set up — beginner-safe.

02

Master prompt

Copy-paste once. Claude tunes itself for tokens.

03

Cheat sheet

20 commands, repos & skills on one page.

⚙️ BEFORE YOU START — YOU NEED CLAUDE CODE INSTALLED

This guide is for the **Claude Code** CLI. Don't have it yet? Install Node 18+, then run `npm install -g @anthropic-ai/claude-code` and start it by typing `claude` in any project folder. Full setup: docs.claude.com/claude-code.

1

Check Claude Code is installed

Open your terminal and type `claude --version`. No version? Install it:
`npm install -g @anthropic-ai/claude-code` (needs Node 18+).

2

Open it inside your project

In your terminal, go to your project folder, then type `claude` and press Enter. You're now in a Claude Code session.

3

Paste the Master Prompt (below)

Copy the whole block on this page, paste it into Claude Code, and press Enter. It writes your token rules into the project so every future session stays lean.

4

Work the way it tells you

Run `/compact` at milestones, `/clear` between unrelated tasks, and check spend with `/usage`. The cheat sheet (page 3) has the full set.

5

Measure it

Run `npm ccusage@latest daily` to see exactly what you burned, in dollars. What gets measured gets cut.

► THE MASTER PROMPT — COPY & PASTE INTO CLAUDE CODE

select all → copy

claude — token-efficiency setup

Set this project up for maximum token efficiency, and keep it that way.

1. If there is no CLAUDE.md yet, run `/init`. Then add a "## Token Rules" section to CLAUDE.md with these standing rules and follow them from now on: - Default to the Sonnet model; only suggest Opus for genuinely deep work. - Before any multi-file or large change, stop and show me a short plan first, then wait for my "go" before editing anything. - Read only the specific lines or sections you need — never re-read a whole file you have already read this session. - Push noisy or exploratory work (wide searches, log reading, big diffs) to a subagent so its output stays out of our main conversation. - After each milestone remind me to run `/compact`; between unrelated tasks remind me to run `/clear`. - Be terse: no filler, no restating my request, no trailing summaries unless I ask for one. 2. Then tell me: the 3 biggest token sinks in how I am working right now, and the single change that would save the most. 3. Confirm exactly what you wrote to CLAUDE.md, then wait for my next instruction.

Tip: This only edits a local CLAUDE.md rules file — it changes nothing else and costs almost no tokens. Run it once per project. Re-run any time your sessions start feeling heavy.

The cheat sheet

02 — 20 TACTICS

① NATIVE COMMANDS — FREE, BUILT IN

CMD /compact

Summarize the chat into a compact state & keep going — frees the context window.

CMD /clear

Wipe context when you switch to unrelated work. Stops old tokens riding along.

CMD Subagents

Delegate noisy jobs — verbose output stays in the subagent, only a summary returns.

CMD /usage

Check live token spend mid-session before you blow the budget.

CMD /effort low

Skip deep reasoning on easy tasks — lower effort, fewer thinking tokens.

CMD Plan mode

Approve the plan once (Shift+Tab) — no wasted billable re-do.

② GITHUB REPOS — INSTALL & RUN

REPO ccusage

See exactly what you burned, in \$, by day/session/project. Local logs, nothing uploaded.

```
npx ccusage@latest daily
```

github.com/ryoppippi/ccusage

REPO Usage-Monitor

Real-time monitor that predicts & warns you BEFORE you hit the rate limit.

```
pip install claude-monitor
```

github.com/Maciek-roboblog/Claude-Code-Usage-Monitor

REPO repomix --compress

Pack a repo into one file. --compress (Tree-sitter) → ~70% fewer tokens.

```
npx repomix --compress
```

github.com/yamadashy/repomix

REPO repomix --token-budget

Hard cap — fails if packed output exceeds N tokens. A guard for agent runs.

```
npx repomix --token-budget 50000
```

③ MEMORY & SKILLS — TRAIN IT ONCE

MEMORY CLAUDE.md

Standing rules auto-loaded each session. Keep tight — every line is paid every session.

```
/init → edit CLAUDE.md
```

MEMORY File-memory

Thin manifest + on-demand notes so Claude recalls projects across sessions.

CMD /context

Visualize what's filling the context window + where to trim it.

SKILL Skills > re-prompt

Repeated workflow → one Skill. Write it once, never re-paste.

SKILL Caveman mode

Terse in/out, drops filler. ~70% lighter I/O, full accuracy.

SKILL /compress memory

Shrink CLAUDE.md / memory files into compact form to cut input tokens.

CONFIG Start on Sonnet

Default Sonnet; only call Opus for genuinely deep refactors.

CONFIG MAX_THINKING_TOKENS

Cap the reasoning budget so thinking can't run away.

```
MAX_THINKING_TOKENS=8000
```

CONFIG Read sections

Read only the lines you need; don't re-read whole files every turn.

The pro stack: Run **ccusage** to see where tokens go → trim **CLAUDE.md** & watch **/context** → push noisy work to **subagents** → **/compact** at every milestone, **/clear** between tasks. That combo alone routinely halves spend before you touch model settings.